Practical Work 3: File Transfer using MPI

Group ID: 16

December 16, 2025

1 Introduction

The objective of this practical work is to upgrade a standard TCP file transfer system into a distributed system using the **Message Passing Interface (MPI)** standard. The goal is to transfer a file from a Sender process to a Receiver process within an MPI environment.

2 MPI Implementation Choice

We chose **Python** with the **mpi4py** library for this implementation.

- **Reasoning:** Python offers high-level readability, and `mpi4py` provides Pythonic bindings for the standard MPI standard (like OpenMPI or MPICH). It allows passing arbitrary Python objects (picklable) as well as efficient direct memory buffers (byte-like objects), making it ideal for prototyping file transfer logic.

3 System Design

3.1 Architecture

The system follows a **Master-Slave** (or Peer-to-Peer) pattern where:

- **Rank 0 (Sender):** Reads the file from the disk and transmits metadata (name, size) followed by data chunks.
- **Rank 1 (Receiver):** Listens for metadata, creates a destination file, and writes the incoming data chunks until an End-of-File (EOF) signal is received.

3.2 Data Flow Diagram

The figure below illustrates the communication protocol between the MPI ranks:

4 Implementation

4.1 Sender Logic (Rank 0)

The sender handles file I/O and non-blocking transmission logic.

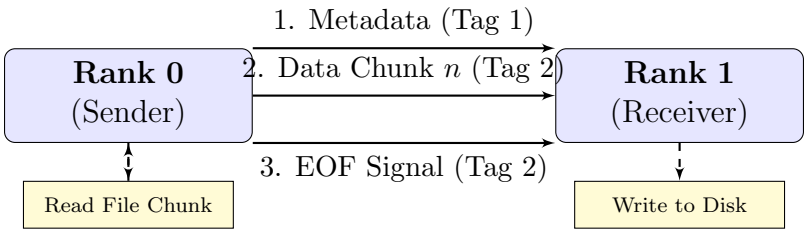

Figure 1: MPI File Transfer Communication Flow

```python
def sender(self):
    # Send metadata
    self.comm.send({'name': self.filename, 'size': size}, dest=1, tag=1)

    # Send content loop
    with open(self.filename, 'rb') as f:
        while True:
            chunk = f.read(self.chunk_size)
            if not chunk: break
            self.comm.send(chunk, dest=1, tag=2)

    # Send termination signal
    self.comm.send(None, dest=1, tag=2)
```

Listing 1: Sender Function

4.2 Receiver Logic (Rank 1)

The receiver waits for specific tags to reconstruct the file.

```python
def receiver(self):
    # Receive metadata
    meta = self.comm.recv(source=0, tag=1)
    new_name = "received_" + meta['name']

    # Receive loop
    with open(new_name, 'wb') as f:
        while True:
            chunk = self.comm.recv(source=0, tag=2)
            if chunk is None: break
            f.write(chunk)
```

Listing 2: Receiver Function

5 Roles and Responsibilities

No.	Member	Task
1	Member 1	MPI Environment Setup, Sender Logic
2	Member 2	Receiver Logic, File I/O Handling
3	Member 3	Report writing, Architecture diagrams

Table 1: Group Roles